Space Shooter — Build Instructions

A complete, self-contained Unity 2D space shooter. All gameplay code, procedural sprites, procedural audio, scenes, prefabs and project settings are included. No external Unity packages or paid assets are required.

---

1. Prerequisites

Requirement	Details
Unity Hub	https://unity.com/download
Unity Editor	**2021.3 LTS** (tested with 2021.3.16f1) or any newer 2021.3.x / 2022.x LTS. Install via Unity Hub.
Build support module	During install, tick **Windows Build Support (IL2CPP)** (and **Windows Build Support (Mono)** as a fallback).
Visual Studio 2019/2022	Optional but recommended for editing/IntelliSense. The “Game development with Unity” workload includes the C# tools.
Python 3.8+	Only needed once, to (re)generate sprites. https://www.python.org/downloads/
Pillow	Python imaging library: `pip install Pillow`

> IL2CPP builds on Windows also require **Visual Studio with the “Desktop development with C++” workload** installed (MSVC toolchain). If you do not have it, use the Mono scripting backend instead (see §7).

---

2. Generate the Sprites (one-time)

The PNG sprites are produced by a Python script so the repository stays lightweight.

```
cd SpaceShooter
pip install Pillow
python GenerateSprites.py
```

This writes 18 PNG files into `Assets/Sprites/`. Re-running overwrites them.

> **Note:** If you received the project with the sprites and `.meta` files already present, you can skip this step. If you regenerate sprites, keep the existing `.meta` files so the import settings and GUID references remain intact.

(Optional) Regenerate scenes/prefabs/meta

The `.meta`, `.prefab` and `.unity` files are already generated and wired. If you ever need to regenerate them (e.g. after adding scripts), run:

```
python generate_unity_assets.py
```

---

3. Open the Project

1. Launch **Unity Hub → Open → Add project from disk**.
2. Select the `SpaceShooter` folder (the one containing `Assets/`, `Packages/`, `ProjectSettings/`).
3. Unity may prompt to upgrade the editor version — accept a 2021.3+ LTS version.
4. First import will take a minute while Unity compiles scripts and imports sprites.

If Unity asks about the Input System backend, choose **Both** (the project's `ProjectSettings.asset` already sets `activeInputHandler: 2` = Both, and all gameplay uses the classic `Input` API defined in `InputManager.asset`).

---

4. Sprite Import Settings

The generated `.meta` files already configure every PNG as a **Sprite (2D and UI)** with:
- Texture Type: **Sprite (2D and UI)**
- Pixels Per Unit: **100**
- Filter Mode: **Point (no filter)** — crisp pixel look
- Compression: **None** / Alpha Is Transparency: **On**

If you regenerated a sprite and lost its `.meta`, select it in the Project window and set:
- **Texture Type** → Sprite (2D and UI)
- **Sprite Mode** → Single (for `explosion_sheet.png` you may switch to **Multiple** and slice into 4×1 frames if you want animated sheets — the code works with a single sprite too)
- **Filter Mode** → Point
- Click **Apply**

---

5. Scenes

Two scenes are provided in `Assets/Scenes/`:

Scene	Purpose
`MainMenu.unity`	Title, Play / Options / Quit, volume sliders, difficulty, high score.
`GameScene.unity`	The playable game: camera + shake, all managers, object pool (pre-wired), parallax background, UI manager and a Player object.

Both scenes are already added to **File → Build Settings → Scenes In Build** (via `ProjectSettings/EditorBuildSettings.asset`), with `MainMenu` at index 0.

Verify Build Settings

1. **File → Build Settings…**
2. Confirm the list shows, in order:
   - `Scenes/MainMenu` (index 0)
   - `Scenes/GameScene` (index 1)
3. If empty, click **Add Open Scenes** while each scene is open, MainMenu first.

---

6. Prefabs & Reference Wiring

All prefabs live in `Assets/Prefabs/` and already have their scripts, sprites, colliders and Rigidbody2D attached. The **ObjectPool** in `GameScene` is pre-wired with every prefab under the matching pool tag, so pooling works on Play with no manual setup:

Pool Tag	Prefab
`PlayerBullet`	PlayerBullet.prefab
`EnemyBullet`	EnemyBullet.prefab
`EnemyA`	EnemyA.prefab
`EnemyB`	EnemyB.prefab
`EnemyBoss`	EnemyBoss.prefab
`Explosion`	Explosion.prefab
`PowerUp_Speed/Rapid/Triple/Shield/Health/Bomb`	matching PowerUp prefabs

Optional polish (recommended, not required to run)

These references are exposed via `[SerializeField]` and can be assigned in the Inspector for extra fidelity. They all degrade gracefully (null-checked) if left empty:

- **Player → PlayerShooter → Fire Point**: create an empty child at the ship's nose and assign it. If empty, bullets spawn at the ship center.
- **Player → PlayerShooter → Laser Line**: add a `LineRenderer` for the Laser power-up beam. If empty, Laser falls back to fast single shots.
- **ParallaxBackground**: assign `star_small` / `star_large` sprites and up to 3 child layer transforms with speed multipliers 0.2 / 0.5 / 1.0 to enable the scrolling star field.
- **UIManager / HUDController / MainMenuController / PauseMenuController / GameOverController**: wire the TextMeshPro texts, Images (health/boss/power-up fill), buttons, sliders and panels described in each script's `[Header]` sections. Use a Canvas (Screen Space – Overlay, Reference Resolution 1920×1080, Scale With Screen Size).

> The game is fully playable from `GameScene` even before wiring the optional UI — the HUD/menu scripts simply skip any element that isn't assigned.

Building a Canvas HUD (quick guide)

1. In `GameScene`, right-click Hierarchy → **UI → Canvas** (this also creates an EventSystem).
2. Canvas Scaler → **Scale With Screen Size**, 1920×1080, Match 0.5.
3. Add child `Image` for the health bar (assign `ui_healthbar`, Image Type → Filled → Horizontal) and drag it into **HUDController → Health Fill**.
4. Add TMP texts for Score / Wave / Combo and drag them into the matching HUDController fields.
5. Add a hidden panel `GameOverPanel` with score/high-score texts and Retry / Main Menu buttons; wire into **GameOverController**.
6. Add a hidden `PausePanel` with Resume / Restart / Main Menu buttons; wire into **PauseMenuController**.
7. Register each panel (name + CanvasGroup + root) in **UIManager → Panels** if you want the fade animations.

---

7. Player Settings

`ProjectSettings/ProjectSettings.asset` already sets:
- **Product Name:** Space Shooter
- **Company Name:** IndieStudio
- **Version:** 1.0.0
- **Default fullscreen mode:** Fullscreen Window (Windowed Fullscreen)
- **Scripting Backend:** IL2CPP (Standalone)
- **Api Compatibility Level:** .NET Standard 2.1

To review/change: **Edit → Project Settings → Player → Other Settings**.

Resolution / Quality

- **Edit → Project Settings → Player → Resolution and Presentation**: default 1920×1080, Windowed Fullscreen.
- **Edit → Project Settings → Quality**: "High" is the default for Standalone/Windows.

Application Icon (optional)

- **Player → Icon**: assign any texture (e.g. `player_ship.png`) as the default icon.

---

8. Building the Windows Executable

1. **File → Build Settings…**
2. **Platform:** select **Windows, Mac, Linux**. Click **Switch Platform** if not already active.
3. **Target Platform:** Windows • **Architecture:** x86_64.
4. **Scripting backend** (Player Settings): **IL2CPP** recommended for a standalone `.exe`.
   - If IL2CPP fails due to a missing C++ toolchain, switch to **Mono** (Player → Other Settings → Scripting Backend → Mono) and rebuild.
5. Click **Build** (or **Build And Run**).
6. Choose an output folder, e.g. `Builds/Windows/`.
7. Unity produces `Space Shooter.exe` plus a `Space Shooter_Data/` folder and `UnityPlayer.dll`. Ship the whole folder together.

---

9. Controls

Action	Keys
Move	**WASD** or **Arrow Keys**
Shoot	**Space** or **Left Mouse**
Pause / Resume	**Escape**

---

10. Testing Checklist

- [ ] MainMenu loads; title bobs; Play starts the game; volume sliders affect audio.
- [ ] Player moves within screen bounds and tilts when moving horizontally.
- [ ] Player fires; bullets travel up and despawn (pooled).
- [ ] Waves spawn enemies in formations; Type A drifts, Type B zigzags & aims.
- [ ] Enemies take damage, explode (particles + sound), sometimes drop power-ups.
- [ ] Power-ups apply effects (speed, rapid, triple, shield, health, bomb clears screen).
- [ ] Score increases; combo multiplier rises with rapid kills.
- [ ] Wave 5 & 10 spawn the boss with a health bar and two attack phases (red tint in phase 2).
- [ ] Taking damage triggers invincibility flashing + camera shake; losing all health costs a life.
- [ ] Losing all lives shows Game Over with animated score count-up and high-score save.
- [ ] Escape pauses (Time.timeScale = 0) and shows the pause menu.

---

11. Troubleshooting

Symptom	Fix
Pink / magenta sprites	Sprite material missing; ensure URP is not half-installed. This project uses the built-in `Sprites/Default` material — remove URP if you added it, or create a 2D URP renderer.
Scripts show "missing" on components	Let Unity finish compiling; check the Console for compile errors and fix them, then reopen the scene. Ensure the `.meta` GUIDs weren't deleted.
No sprites in Project	Run `python GenerateSprites.py`, then in Unity **Assets → Reimport All**.
No enemies spawn	Confirm the **ObjectPool** GameObject in `GameScene` still has its `pools` list populated (12 entries). Re-run `python generate_unity_assets.py` if needed.
No audio	Audio is procedural; ensure an **AudioListener** exists (it's on the Main Camera) and master/SFX/music volumes aren't at 0 in the options menu.
IL2CPP build error about C++	Install Visual Studio "Desktop development with C++", or switch Scripting Backend to **Mono**.
"Input Axis Fire1 not set up"	The project ships an `InputManager.asset` with Horizontal, Vertical, Fire1 and Cancel. If missing, reimport `ProjectSettings/`.
Blurry sprites	Set the sprite's Filter Mode to **Point (no filter)** and Compression to **None**.
Scenes not in build	File → Build Settings → drag `MainMenu` then `GameScene` into "Scenes In Build".

12. Project Layout

```
SpaceShooter/
├── Assets/
│   ├── Scripts/        # All C# gameplay code (Core, Player, Enemy, Weapons, Systems,
UI, Effects)
│   ├── Scenes/         # MainMenu.unity, GameScene.unity
│   ├── Prefabs/        # Player, enemies, bullets, explosion, power-ups (.prefab)
│   ├── Sprites/        # PNGs generated by GenerateSprites.py
│   └── Audio/          # (procedural audio at runtime; no files needed)
├── Packages/manifest.json
├── ProjectSettings/    # ProjectSettings, InputManager, TagManager, QualitySettings,
etc.
├── GenerateSprites.py
├── generate_unity_assets.py   # regenerates .meta / prefabs / scenes if needed
└── BUILD_INSTRUCTIONS.md
```

Enjoy, and happy shooting! 🚀